Casos de prueba de integración

> Generado desde `scripts/casos_de_prueba.py` (fuente única). Para registrar resultados usa `casos_de_prueba.xlsx` (columnas *Estado*, *Resultado real*, *Evidencia*).

Total: 75 casos — 33 automatizados (pytest), 12 semi-automáticos, 30 manuales.

Tipo: A = automatizado · S = semi-automático (script/simulador + revisión) · M = manual.
Prioridad: Alta = bloquea la aceptación; Media = debe cumplirse; Baja = deseable.

Resumen por nivel

Nivel	Componente	Casos	Alta	A	S	M
N0	Entorno AWS y Docker	8	7	0	3	5
N1	Sensor ↔ ESP32	5	2	0	0	5
N2	ESP32 ↔ Red WiFi / NTP	5	2	0	0	5
N2	ESP32 ↔ API (contrato HTTP)	21	9	17	0	4
N3	API ↔ Base de datos	11	6	9	1	1
N4	Extremo a extremo	7	3	0	3	4
N5	No funcionales	9	4	2	3	4
N6	Seguridad básica	9	4	5	2	2

Requisitos y trazabilidad

Requisito	Descripción	Casos
RF-01	El sistema mide temperatura y humedad con el sensor conectado al ESP32.	IT-SEN-01, IT-SEN-03, IT-SEN-04, IT-SEN-05
RF-02	El ESP32 se conecta a la red WiFi y recupera la conexión tras una caída.	IT-NET-01, IT-NET-02, IT-NET-03
RF-03	El ESP32 envía cada lectura a la API en JSON, autenticado con API key.	IT-ENV-02, IT-API-18
RF-04	La API valida el contenido y lo persiste en la base SQL.	IT-ENV-04, IT-API-01, IT-API-05, IT-API-07, IT-API-08, IT-API-10, IT-DB-01, IT-DB-05, IT-DB-06, IT-DB-07, IT-DB-10, IT-E2E-02
RF-05	No se pierden ni se duplican lecturas ante reintentos, cortes o reinicios.	IT-SEN-02, IT-API-02, IT-API-12, IT-API-19, IT-API-20, IT-DB-02, IT-DB-03, IT-E2E-01, IT-E2E-03, IT-E2E-04, IT-NFR-02, IT-NFR-08
RF-06	Las lecturas pueden consultarse por dispositivo y rango de tiempo.	IT-API-14, IT-API-15, IT-E2E-06
RF-07	Se pueden agregar nuevas magnitudes (p. ej. PM2.5) sin cambiar el esquema.	IT-DB-04, IT-E2E-07

Requisito	Descripción	Casos
RF-08	Un dashboard de Grafana muestra las lecturas almacenadas (temperatura, humedad, estado de los dispositivos).	IT-ENV-07
RNF-01	Latencia de entrega (ESP32 → BD) con p95 menor a 2 s en condiciones normales.	IT-NFR-01, IT-NFR-03, IT-NFR-04
RNF-02	El sistema se recupera solo tras caídas de red, API, BD o energía.	IT-ENV-01, IT-ENV-05, IT-ENV-06, IT-ENV-08, IT-NET-05, IT-API-17, IT-API-21, IT-DB-08, IT-DB-11, IT-NFR-05, IT-NFR-06, IT-NFR-07, IT-NFR-09
RNF-03	Seguridad básica: autenticación, validación de entradas, BD no expuesta, sin secretos en el código.	IT-ENV-03, IT-API-03, IT-API-04, IT-API-06, IT-API-09, IT-API-13, IT-API-16, IT-SEC-01, IT-SEC-02, IT-SEC-03, IT-SEC-04, IT-SEC-05, IT-SEC-06, IT-SEC-07, IT-SEC-08, IT-SEC-09
RNF-04	Trazabilidad temporal: cada lectura tiene hora del dispositivo (NTP) y hora del servidor.	IT-NET-04, IT-API-11, IT-DB-09, IT-E2E-05

N0 — Entorno AWS y Docker

IT-ENV-01 — Acceso SSH a la EC2 y versión de Python

- **Requisito:** RNF-02 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** Laboratorio iniciado; EC2 en ejecución.
- **Pasos:** 1. ssh -i labsuser.pem ec2-user@ 2. python3 --version
- **Datos:** IP pública / Elastic IP
- **Resultado esperado:** Acceso correcto; Python ≥ 3.11 disponible.

IT-ENV-02 — Puerto de la API alcanzable desde la red del ESP32

- **Requisito:** RF-03 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** SG `sg-api` con regla de entrada TCP 8000; API en ejecución.
- **Pasos:** 1. Desde un celular con datos móviles y desde el WiFi del ESP32: abrir http://:8000/health
- **Datos:** URL /health
- **Resultado esperado:** Responde 200 con {"status":"ok","db":"ok"} desde ambas redes.

IT-ENV-03 — Stack docker compose sano y PostgreSQL sin puertos publicados

- **Requisito:** RNF-03 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** EC2 con Docker; `deploy/.env` creado.
- **Pasos:** 1. En la EC2: cd deploy && docker compose up -d --build 2. docker compose ps (todos healthy) 3. docker compose exec db psql -U sensores_app -d sensores -c 'SELECT 1' 4. docker compose port db 5432
- **Datos:** deploy/docker-compose.yml
- **Resultado esperado:** api y db en estado healthy; `SELECT 1` devuelve 1; `docker compose port db 5432` no devuelve ningún puerto publicado.

IT-ENV-04 — Esquema aplicado con restricciones

- **Requisito:** RF-04 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** Contenedor `db` iniciado con el volumen vacío (aplica `db/schema.sql` al inicializar).
- **Pasos:** 1. docker compose exec db psql -U sensores_app -d sensores 2. En psql: \d lectura 3. Verificar UNIQUE (dispositivo_id, seq) y FKs con ON DELETE CASCADE 4. Verificar que existe el rol grafana_ro con solo SELECT
- **Datos:** db/schema.sql · db/02-roles.sh
- **Resultado esperado:** Existen dispositivo, lectura y medicion con índices y restricciones.

IT-ENV-05 — Contenedores con política de reinicio

- **Requisito:** RNF-02 · **Prioridad:** Media · **Tipo:** Manual
- **Precondiciones:** Stack en ejecución.
- **Pasos:** 1. docker inspect -f '{{.HostConfig.RestartPolicy.Name}}' nextguard-api-1 nextguard-db-1 nextguard-grafana-1 2. curl localhost:8000/health
- **Datos:** —
- **Resultado esperado:** Los tres contenedores usan `unless-stopped`; /health = 200.

IT-ENV-06 — Prueba de humo del entorno

- **Requisito:** RNF-02 · **Prioridad:** Alta · **Tipo:** Semi-automático
- **Precondiciones:** Stack docker compose operativo.
- **Pasos:** 1. En la EC2: API_URL=http://localhost:8000 API_KEY=… GRAFANA_URL=http://localhost:3000 ./scripts/smoke_env.sh
- **Datos:** smoke_env.sh
- **Resultado esperado:** Todas las verificaciones [OK], código de salida 0.
- **Automatización:** `scripts/smoke_env.sh`

IT-ENV-07 — Grafana con datasource y dashboard aprovisionados

- **Requisito:** RF-08 · **Prioridad:** Alta · **Tipo:** Semi-automático
- **Precondiciones:** Stack en ejecución; lecturas enviadas con el simulador.
- **Pasos:** 1. Abrir http://:3000 e iniciar sesión con el usuario de .env 2. Connections → Data sources → NextGuard → Save & test 3. Abrir el dashboard «NextGuard · Monitoreo IoT»
- **Datos:** deploy/grafana/
- **Resultado esperado:** El datasource responde «Database Connection OK»; los paneles de temperatura, humedad y lecturas por minuto muestran las lecturas del simulador.

IT-ENV-08 — Persistencia de datos tras bajar y subir el stack

- **Requisito:** RNF-02 · **Prioridad:** Alta · **Tipo:** Semi-automático
- **Precondiciones:** Lecturas cargadas en la BD.

- **Pasos:** 1. docker compose down (sin -v) 2. docker compose up -d 3. GET /api/v1/lecturas y abrir el dashboard
- **Datos:** —
- **Resultado esperado:** Las lecturas y el dashboard siguen disponibles; la API reconecta sola a la BD.

N1 — Sensor ↔ ESP32

IT-SEN-01 — Lectura DHT válida y dentro de rango

- **Requisito:** RF-01 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** USE_DHT=1; DHT cableado (VCC, GND, DATA con pull-up); monitor serie a 115200.
- **Pasos:** 1. Cargar firmware 2. Observar 10 muestras en el monitor serie 3. Comparar con un termómetro/higrómetro de referencia
- **Datos:** DHT11 o DHT22
- **Resultado esperado:** T entre -40 y 80 °C y H entre 0 y 100 %; diferencia con la referencia dentro de la tolerancia del sensor (DHT22 ±0,5 °C; DHT11 ±2 °C).

IT-SEN-02 — Lectura inválida (NaN) no se envía y el sistema sigue

- **Requisito:** RF-05 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** Firmware ejecutándose con DHT real.
- **Pasos:** 1. Desconectar el pin DATA durante 30 s 2. Reconectarlo
- **Datos:** —
- **Resultado esperado:** Se imprime '[SENSOR] lectura inválida' y no se encola nada; al reconectar se retoman las lecturas sin reiniciar.

IT-SEN-03 — Frecuencia de muestreo

- **Requisito:** RF-01 · **Prioridad:** Media · **Tipo:** Manual
- **Precondiciones:** Firmware ejecutándose.
- **Pasos:** 1. Registrar durante 5 min las marcas de tiempo de [MUESTRA] 2. Calcular el periodo medio
- **Datos:** INTERVALO_MS = 10000
- **Resultado esperado:** Periodo medio 10 s ±5 %; sin ráfagas ni huecos.

IT-SEN-04 — Respuesta a un cambio físico

- **Requisito:** RF-01 · **Prioridad:** Media · **Tipo:** Manual
- **Precondiciones:** Firmware ejecutándose y llegando datos a la BD.
- **Pasos:** 1. Exhalar sobre el sensor 5 s 2. Consultar las últimas lecturas por API
- **Datos:** —
- **Resultado esperado:** La humedad sube de forma visible y el cambio aparece en la BD.

IT-SEN-05 — Modo simulado sin hardware

- **Requisito:** RF-01 · **Prioridad:** Baja · **Tipo:** Manual
- **Precondiciones:** USE_DHT=0.
- **Pasos:** 1. Cargar firmware 2. Observar 20 muestras
- **Datos:** —
- **Resultado esperado:** Valores plausibles (T ≈ 22 °C variando ≤ 0,3 °C por muestra; H entre 0 y 100 %).

N2 — ESP32 ↔ Red WiFi / NTP

IT-NET-01 — Conexión WiFi inicial e IP

- **Requisito:** RF-02 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** Red 2.4 GHz con WPA2; config.h correcto.
- **Pasos:** 1. Encender el ESP32 2. Observar el monitor serie
- **Datos:** SSID/clave
- **Resultado esperado:** Se imprime la IP asignada en menos de 20 s.
- **Automatización:** `wifi_diagnostico.ino`

IT-NET-02 — Clave WiFi incorrecta: motivo registrado y sin bloqueo

- **Requisito:** RF-02 · **Prioridad:** Media · **Tipo:** Manual
- **Precondiciones:** config.h con clave errónea.
- **Pasos:** 1. Cargar firmware 2. Observar 2 min
- **Datos:** Clave inválida
- **Resultado esperado:** Se registra el motivo de desconexión; el muestreo continúa y las muestras se encolan (sin cuelgue ni reinicio).

IT-NET-03 — Caída del router durante la ejecución

- **Requisito:** RF-02 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** ESP32 enviando con normalidad.
- **Pasos:** 1. Apagar el router 2 min 2. Encenderlo 3. Esperar 3 min
- **Datos:** —
- **Resultado esperado:** El ESP32 se reconecta solo; las muestras acumuladas se entregan (sin faltas ni duplicados).

IT-NET-04 — Hora NTP y campo ts_dispositivo

- **Requisito:** RNF-04 · **Prioridad:** Media · **Tipo:** Manual
- **Precondiciones:** Internet disponible.
- **Pasos:** 1. Arrancar y revisar '[MUESTRA] … hora=NTP' 2. Consultar la API y ver ts_dispositivo 3. Repetir con Internet bloqueado en el router
- **Datos:** —

- **Resultado esperado:** Con NTP: ts_dispositivo en UTC correcto. Sin NTP: ts_dispositivo = null y la lectura se guarda igual.

IT-NET-05 — Entrega según intensidad de señal

- **Requisito:** RNF-02 · **Prioridad:** Baja · **Tipo:** Manual
- **Precondiciones:** Dispositivo alimentado por batería/USB estable.
- **Pasos:** 1. Medir 10 min a tres distancias (RSSI ≈ -50, -70, -80 dBm) 2. Comparar enviadas vs guardadas
- **Datos:** RSSI
- **Resultado esperado:** Entrega ≥ 99 % con RSSI ≥ -80 dBm.

N2 — ESP32 ↔ API (contrato HTTP)

IT-API-01 — Crear lectura válida → 201

- **Requisito:** RF-04 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** API con API key válida.
- **Pasos:** 1. POST /api/v1/lecturas con payload válido
- **Datos:** payload estándar
- **Resultado esperado:** 201 y {duplicado:false}.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_01`

IT-API-02 — Reintento con el mismo seq → 200 idempotente

- **Requisito:** RF-05 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** Lectura seq=7 ya guardada.
- **Pasos:** 1. Reenviar la misma lectura
- **Datos:** seq=7
- **Resultado esperado:** 200 con {duplicado:true}; sigue habiendo 1 sola fila.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_02`

IT-API-03 — Sin API key → 401

- **Requisito:** RNF-03 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST sin cabecera X-API-Key
- **Datos:** —
- **Resultado esperado:** 401 y no se guarda nada.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_03`

IT-API-04 — API key incorrecta → 401

- **Requisito:** RNF-03 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —

- **Pasos:** 1. POST con X-API-Key errónea
- **Datos:** —
- **Resultado esperado:** 401.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_04`

IT-API-05 — Lectura sin mediciones → 422

- **Requisito:** RF-04 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST con mediciones=[]
- **Datos:** —
- **Resultado esperado:** 422.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_05`

IT-API-06 — device_id con caracteres inválidos → 422

- **Requisito:** RNF-03 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST con device_id='esp 01;'
- **Datos:** —
- **Resultado esperado:** 422.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_06`

IT-API-07 — Valor NaN → 422 (no 500)

- **Requisito:** RF-04 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST con "valor": NaN (JSON no estándar)
- **Datos:** —
- **Resultado esperado:** 422 con detalle sin el valor recibido; la API no falla con 500.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_07`

IT-API-08 — JSON malformado → 422

- **Requisito:** RF-04 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST con cuerpo '{no es json'
- **Datos:** —
- **Resultado esperado:** 422.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_08`

IT-API-09 — Payload demasiado grande → 413

- **Requisito:** RNF-03 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** MAX_BODY_BYTES configurado.
- **Pasos:** 1. POST con cuerpo mayor al límite

- **Datos:** ≥ 5 KB en pruebas
- **Resultado esperado:** 413.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_09`

IT-API-10 — Campo desconocido → 422

- **Requisito:** RF-04 · **Prioridad:** Baja · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST con un campo extra
- **Datos:** —
- **Resultado esperado:** 422 (esquema estricto).
- **Automatización:** `tests/test_api_contrato.py::test_it_api_10`

IT-API-11 — Sin ts_dispositivo se acepta

- **Requisito:** RNF-04 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST sin ts_dispositivo 2. GET de la lectura
- **Datos:** —
- **Resultado esperado:** 201; ts_dispositivo = null.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_11`

IT-API-12 — Lote con nuevas y duplicadas

- **Requisito:** RF-05 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** seq=1 ya guardado.
- **Pasos:** 1. POST /batch con seq 1, 2, 3
- **Datos:** 3 lecturas
- **Resultado esperado:** {insertadas:2, duplicadas:1}.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_12`

IT-API-13 — Lote de más de 100 → rechazado

- **Requisito:** RNF-03 · **Prioridad:** Baja · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST /batch con 101 lecturas
- **Datos:** 101 lecturas
- **Resultado esperado:** 413 o 422.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_13`

IT-API-14 — GET filtra por dispositivo y limit

- **Requisito:** RF-06 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** Lecturas de 2 dispositivos.
- **Pasos:** 1. GET ?device_id=… 2. GET ?limit=2
- **Datos:** —

- **Resultado esperado:** Cantidades exactas por filtro.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_14`

IT-API-15 — GET devuelve mediciones completas

- **Requisito:** RF-06 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** Lectura guardada.
- **Pasos:** 1. GET ?device_id=...
- **Datos:** —
- **Resultado esperado:** Magnitudes, valores, rssi y seq idénticos a lo enviado.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_15`

IT-API-16 — GET requiere API key

- **Requisito:** RNF-03 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. GET sin X-API-Key
- **Datos:** —
- **Resultado esperado:** 401.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_16`

IT-API-17 — /health sin clave y refleja la BD

- **Requisito:** RNF-02 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. GET /health
- **Datos:** —
- **Resultado esperado:** 200 {status:ok, db:ok}.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_17`

IT-API-18 — ESP32 real: primer envío correcto

- **Requisito:** RF-03 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** API accesible; config.h con API_HOST y API_KEY correctos.
- **Pasos:** 1. Encender el ESP32 2. Esperar la primera muestra 3. Ver el monitor serie
- **Datos:** —
- **Resultado esperado:** Se imprime '[API] 201 OK' y la lectura aparece en GET /api/v1/lecturas.

IT-API-19 — ESP32 con API key errónea: 401, backoff y sin pérdida

- **Requisito:** RF-05 · **Prioridad:** Media · **Tipo:** Manual
- **Precondiciones:** config.h con API_KEY incorrecta.
- **Pasos:** 1. Encender 3 min 2. Corregir API_KEY y volver a cargar
- **Datos:** —

- **Resultado esperado:** Se registra 'fallo 401' con espera creciente (2, 4, 8… hasta 60 s); las muestras se conservan en la cola mientras no se llene.

IT-API-20 — API caída 3 min: la cola se vacía al recuperar

- **Requisito:** RF-05 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** ESP32 enviando con normalidad.
- **Pasos:** 1. sudo systemctl stop sensores-api (3 min) 2. sudo systemctl start sensores-api 3. Comparar seq del monitor serie vs BD
- **Datos:** —
- **Resultado esperado:** Sin faltas ni duplicados; se usan envíos por lote para vaciar la cola; el orden se conserva.

IT-API-21 — Timeout de red: el ESP32 no se bloquea

- **Requisito:** RNF-02 · **Prioridad:** Media · **Tipo:** Manual
- **Precondiciones:** ESP32 enviando con normalidad.
- **Pasos:** 1. Quitar la regla del SG que permite el puerto 8000 (los paquetes se descartan) 2. Observar 2 min 3. Restaurar la regla
- **Datos:** —
- **Resultado esperado:** Cada intento termina por timeout (≈5 s), el muestreo sigue a su ritmo y al restaurar se entrega todo.

N3 — API ↔ Base de datos

IT-DB-01 — Inserta dispositivo, lectura y mediciones

- **Requisito:** RF-04 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** BD vacía.
- **Pasos:** 1. POST de una lectura con 2 mediciones 2. Contar filas
- **Datos:** ubicación='Laboratorio 2'
- **Resultado esperado:** 1 dispositivo, 1 lectura, 2 mediciones.
- **Automatización:** `tests/test_bd.py::test_it_db_01`

IT-DB-02 — UNIQUE (dispositivo_id, seq) a nivel de BD

- **Requisito:** RF-05 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** Lectura seq=1 guardada.
- **Pasos:** 1. Insertar otra fila con el mismo dispositivo y seq directamente en la BD
- **Datos:** —
- **Resultado esperado:** IntegrityError (la BD protege contra duplicados aunque falle la API).
- **Automatización:** `tests/test_bd.py::test_it_db_02`

IT-DB-03 — Mismo seq en dispositivos distintos es válido

- **Requisito:** RF-05 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST seq=1 para esp32-01 y esp32-02
- **Datos:** —
- **Resultado esperado:** Ambos 201.
- **Automatización:** `tests/test_bd.py::test_it_db_03`

IT-DB-04 — Nueva magnitud sin cambiar el esquema

- **Requisito:** RF-07 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST con pm2_5 y pm10 además de T/H 2. GET
- **Datos:** pm2_5=12.5; pm10=20.1
- **Resultado esperado:** Las 4 magnitudes se guardan y se consultan.
- **Automatización:** `tests/test_bd.py::test_it_db_04`

IT-DB-05 — Borrado en cascada

- **Requisito:** RF-04 · **Prioridad:** Baja · **Tipo:** Automatizado (pytest)
- **Precondiciones:** Datos cargados.
- **Pasos:** 1. Borrar un dispositivo
- **Datos:** —
- **Resultado esperado:** Se eliminan sus lecturas y mediciones.
- **Automatización:** `tests/test_bd.py::test_it_db_05`

IT-DB-06 — seq negativo rechazado

- **Requisito:** RF-04 · **Prioridad:** Baja · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST con seq=-1
- **Datos:** —
- **Resultado esperado:** 422 antes de llegar a la BD.
- **Automatización:** `tests/test_bd.py::test_it_db_06`

IT-DB-07 — Valores extremos sin pérdida

- **Requisito:** RF-04 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST T=-40, H=100 2. GET
- **Datos:** —
- **Resultado esperado:** Valores idénticos.
- **Automatización:** `tests/test_bd.py::test_it_db_07`

IT-DB-08 — /health devuelve 503 si la BD no responde

- **Requisito:** RNF-02 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. Iniciar la API con una BD inaccesible 2. GET /health
- **Datos:** —
- **Resultado esperado:** 503 y db='sin conexión'.
- **Automatización:** `tests/test_bd.py::test_it_db_08`

IT-DB-09 — Zonas horarias normalizadas a UTC

- **Requisito:** RNF-04 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST con ts_dispositivo con offset -03:00 2. GET
- **Datos:** 2026-09-26T13:00:00-03:00
- **Resultado esperado:** Se almacena y devuelve como 16:00 UTC.
- **Automatización:** `tests/test_bd.py::test_it_db_09`

IT-DB-10 — Suite pytest contra PostgreSQL (contenedor)

- **Requisito:** RF-04 · **Prioridad:** Alta · **Tipo:** Semi-automático
- **Precondiciones:** Contenedor `db` en ejecución; BD de pruebas vacía; psycopg instalado.
- **Pasos:** 1. docker compose exec db createdb -U sensores_app sensores_test 2. TEST_DATABASE_URL=postgresql+psycopg://sensores_app:@localhost:5432/sensores_test pytest -m 'not carga' (con el puerto 5432 publicado temporalmente, o ejecutando pytest dentro del contenedor api)
- **Datos:** BD `sensores_test`
- **Resultado esperado:** Todas las pruebas pasan igual que en SQLite (mismos resultados en PostgreSQL).
- **Automatización:** `tests`

IT-DB-11 — Contenedor db detenido y recuperación sin reiniciar la API

- **Requisito:** RNF-02 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** API y ESP32 funcionando.
- **Pasos:** 1. docker compose stop db durante 5 min 2. GET /health y observar el ESP32 3. docker compose start db 4. Observar
- **Datos:** —
- **Resultado esperado:** Durante la caída: /health 503 y POST con 5xx (el ESP32 conserva la cola). Al volver, la API reconecta sola (pool_pre_ping) y se entrega la cola sin duplicados.

N4 — Extremo a extremo

IT-E2E-01 — Camino feliz: conteo exacto

- **Requisito:** RF-05 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** Entorno completo; ESP32 real.
- **Pasos:** 1. Dejar correr 10 min 2. Contar seq impresos en el monitor serie 3. Contar filas por device_id en la BD
- **Datos:** ≈ 60 lecturas
- **Resultado esperado:** Filas en la BD = muestras enviadas: 0 faltantes y 0 duplicados.

IT-E2E-02 — Coherencia de valores serie ↔ BD

- **Requisito:** RF-04 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** Caso E2E-01 ejecutado.
- **Pasos:** 1. Comparar 10 lecturas al azar (seq, T, H)
- **Datos:** —
- **Resultado esperado:** Diferencia ≤ 0,1 (redondeo a 1 decimal).

IT-E2E-03 — Reinicio del ESP32: seq no se reutiliza

- **Requisito:** RF-05 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** ESP32 enviando.
- **Pasos:** 1. Anotar el último seq 2. Pulsar EN (reset) 3. Comparar el nuevo seq inicial
- **Datos:** —
- **Resultado esperado:** El nuevo seq es mayor (salto de hasta 20 por diseño); no hay lecturas sobrescritas ni duplicadas.

IT-E2E-04 — Dos dispositivos simultáneos

- **Requisito:** RF-05 · **Prioridad:** Media · **Tipo:** Semi-automático
- **Precondiciones:** ESP32 real activo.
- **Pasos:** 1. python simulador_sensor.py --devices 1 --count 60 --interval 10 --verify (mientras corre el ESP32)
- **Datos:** —
- **Resultado esperado:** Cada dispositivo conserva su propia secuencia; sin interferencia.
- **Automatización:** `scripts/simulador_sensor.py`

IT-E2E-05 — Trazabilidad temporal

- **Requisito:** RNF-04 · **Prioridad:** Media · **Tipo:** Manual
- **Precondiciones:** NTP funcionando.
- **Pasos:** 1. Consultar 20 lecturas 2. Calcular $|ts_{servidor} - ts_{dispositivo}|$
- **Datos:** —
- **Resultado esperado:** Diferencia < 60 s (en operación normal).

IT-E2E-06 — Consulta por rango de fechas

- **Requisito:** RF-06 · **Prioridad:** Media · **Tipo:** Semi-automático
- **Precondiciones:** ≥ 30 min de datos.
- **Pasos:** 1. GET ?desde=...&hasta=... con un rango de 5 min
- **Datos:** —
- **Resultado esperado:** Solo lecturas dentro del rango, ordenadas de la más reciente.

IT-E2E-07 — Nueva magnitud extremo a extremo

- **Requisito:** RF-07 · **Prioridad:** Media · **Tipo:** Semi-automático
- **Precondiciones:** API en EC2.
- **Pasos:** 1. python simulador_sensor.py --with-pm --devices 1 --count 10 --verify
- **Datos:** pm2_5
- **Resultado esperado:** Se guarda y se consulta sin migrar el esquema.
- **Automatización:** `scripts/simulador_sensor.py`

N5 — No funcionales

IT-NFR-01 — Latencia local: 200 envíos, p95 < 0,5 s

- **Requisito:** RNF-01 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** SQLite local.
- **Pasos:** 1. Ejecutar la prueba de carga ligera
- **Datos:** 200 envíos
- **Resultado esperado:** p95 < 0,5 s.
- **Automatización:** `tests/test_seguridad_y_carga.py::test_it_nfr_01`

IT-NFR-02 — Reintentos masivos sin duplicados

- **Requisito:** RF-05 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. Enviar 20 seq × 5 veces
- **Datos:** —
- **Resultado esperado:** Exactamente 20 filas.
- **Automatización:** `tests/test_seguridad_y_carga.py::test_it_nfr_02`

IT-NFR-03 — Latencia real hacia AWS (p95 < 2 s)

- **Requisito:** RNF-01 · **Prioridad:** Alta · **Tipo:** Semi-automático
- **Precondiciones:** API en EC2; simulador ejecutado desde fuera de AWS.
- **Pasos:** 1. python simulador_sensor.py --url http://:8000 --devices 3 --count 100 --interval 1 --parallel --verify
- **Datos:** —
- **Resultado esperado:** p95 < 2 s; 0 errores de red; verificación OK.

- **Automatización:** `scripts/simulador_sensor.py`

IT-NFR-04 — Carga: 20 dispositivos × 10 min

- **Requisito:** RNF-01 · **Prioridad:** Media · **Tipo:** Semi-automático
- **Precondiciones:** Entorno AWS completo.
- **Pasos:** 1. python simulador_sensor.py --devices 20 --count 600 --interval 1 --parallel --verify 2. Observar CPU/memoria de la EC2 (top) y conexiones de PostgreSQL (docker stats)
- **Datos:** 20 dispositivos, 1 lectura/s
- **Resultado esperado:** 0 respuestas 5xx; verificación OK; CPU EC2 < 80 %.
- **Automatización:** `scripts/simulador_sensor.py`

IT-NFR-05 — Soak de 3 h con ESP32 real

- **Requisito:** RNF-02 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** Alimentación estable; sesión del laboratorio con ≥ 3,5 h restantes.
- **Pasos:** 1. Dejar funcionando 3 h 2. Cada 30 min revisar el monitor serie 3. Al final: comparar enviadas vs BD y buscar saltos de seq ≥ 20 (reinicios)
- **Datos:** ≈ 1080 lecturas
- **Resultado esperado:** Entrega ≥ 99 %, 0 duplicados, 0 reinicios inesperados.

IT-NFR-06 — Corte de energía del ESP32

- **Requisito:** RNF-02 · **Prioridad:** Media · **Tipo:** Manual
- **Precondiciones:** ESP32 enviando.
- **Pasos:** 1. Desconectar la alimentación 1 min 2. Reconectar
- **Datos:** —
- **Resultado esperado:** Lo ya guardado permanece íntegro; se reanuda solo. Se documenta la pérdida máxima (cola en RAM ≤ 60 muestras).

IT-NFR-07 — Reinicio de la EC2

- **Requisito:** RNF-02 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** Elastic IP asignada.
- **Pasos:** 1. Reiniciar la instancia 2. Esperar el arranque 3. Observar el ESP32
- **Datos:** —
- **Resultado esperado:** Los contenedores arrancan solos (restart: unless-stopped) y el ESP32 reanuda el envío. Sin Elastic IP la IP cambia: se documenta como riesgo R-01.

IT-NFR-08 — Cola llena tras 15 min sin red

- **Requisito:** RF-05 · **Prioridad:** Media · **Tipo:** Manual
- **Precondiciones:** Firmware con COLA_MAX=60 e INTERVALO=10 s.
- **Pasos:** 1. Apagar el router 15 min 2. Encenderlo 3. Comparar con la BD
- **Datos:** —

- **Resultado esperado:** Se conservan las últimas 60 muestras, las más antiguas se descartan y el contador `perdidas` coincide con las faltantes en la BD.

IT-NFR-09 — Crecimiento de la BD

- **Requisito:** RNF-02 · **Prioridad:** Baja · **Tipo:** Semi-automático
- **Precondiciones:** ≥ 10 000 lecturas cargadas con el simulador.
- **Pasos:** 1. Consultar pg_total_relation_size para las 3 tablas
- **Datos:** —
- **Resultado esperado:** Se documenta el tamaño por 1000 lecturas y la proyección a 30 días para 10 dispositivos.

N6 — Seguridad básica

IT-SEC-01 — Inyección SQL en device_id

- **Requisito:** RNF-03 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST con device_id="x'; DROP TABLE lectura;--"
- **Datos:** —
- **Resultado esperado:** 422; las tablas siguen intactas.
- **Automatización:** `tests/test_seguridad_y_carga.py::test_it_sec_01`

IT-SEC-02 — Inyección SQL en texto libre

- **Requisito:** RNF-03 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST con ubicacion="'); DROP TABLE dispositivo;--"
- **Datos:** —
- **Resultado esperado:** Se guarda literalmente; las tablas siguen intactas.
- **Automatización:** `tests/test_seguridad_y_carga.py::test_it_sec_02`

IT-SEC-03 — Inyección SQL en filtros GET

- **Requisito:** RNF-03 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. GET ?device_id=a' OR '1'='1
- **Datos:** —
- **Resultado esperado:** 422.
- **Automatización:** `tests/test_seguridad_y_carga.py::test_it_sec_03`

IT-SEC-04 — Magnitud con caracteres no permitidos

- **Requisito:** RNF-03 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —

- **Pasos:** 1. POST con magnitud='temp; DROP'
- **Datos:** —
- **Resultado esperado:** 422.
- **Automatización:** `tests/test_seguridad_y_carga.py::test_it_sec_04`

IT-SEC-05 — Sin claves configuradas se rechaza todo

- **Requisito:** RNF-03 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. Iniciar la API con API_KEYS vacío 2. POST con cualquier clave
- **Datos:** —
- **Resultado esperado:** 401 (falla cerrada).
- **Automatización:** `tests/test_seguridad_y_carga.py::test_it_sec_05`

IT-SEC-06 — PostgreSQL no accesible desde Internet

- **Requisito:** RNF-03 · **Prioridad:** Alta · **Tipo:** Semi-automático
- **Precondiciones:** Compose sin `ports` en el servicio db; SG sin regla para 5432.
- **Pasos:** 1. Desde tu PC (fuera de AWS): EC2_HOST= ./scripts/smoke_env.sh --externo
- **Datos:** —
- **Resultado esperado:** El puerto 5432 no responde desde fuera.
- **Automatización:** `scripts/smoke_env.sh --externo`

IT-SEC-07 — Sin secretos en el código

- **Requisito:** RNF-03 · **Prioridad:** Alta · **Tipo:** Semi-automático
- **Precondiciones:** —
- **Pasos:** 1. grep -rniE "(password|api_key|secret)\s*[=:]" . --include=*.ino --include=*.py --include=*.h --include=*.md 2. Verificar que config.h y .env están en .gitignore
- **Datos:** —
- **Resultado esperado:** Solo aparecen valores de ejemplo (`CAMBIAR`, `TU_CLAVE`); ningún secreto real.

IT-SEC-08 — La API key no aparece en los logs

- **Requisito:** RNF-03 · **Prioridad:** Media · **Tipo:** Manual
- **Precondiciones:** Contenedor api en ejecución.
- **Pasos:** 1. Enviar lecturas con clave válida y errónea 2. docker compose logs api | grep -i
- **Datos:** —
- **Resultado esperado:** Sin coincidencias.

IT-SEC-09 — Tráfico en claro vs HTTPS (fase 2)

- **Requisito:** RNF-03 · **Prioridad:** Baja · **Tipo:** Manual
- **Precondiciones:** Captura con Wireshark en la red del ESP32.
- **Pasos:** 1. Capturar con HTTP 2. Repetir con USE_HTTPS=1

- **Datos:** —
- **Resultado esperado:** Con HTTP se ve la X-API-Key en claro (riesgo aceptado en laboratorio, documentado); con HTTPS no.